LangGraph Guide (official docs)

Contents

LangGraph overview **2**
- Install . . . 2
- Capabilities . . . 2
- Integrations, deployment, and observability . . . 3
- See also . . . 3

Graph API overview **4**
- Graphs . . . 4
  - Super-steps . . . 4
  - StateGraph . . . 4
  - Compiling your graph . . . 4
- State . . . 5
  - Schema . . . 5
  - Reducers . . . 5
  - Working with messages in graph state . . . 6
- Nodes . . . 7
  - Re-execution and idempotency . . . 8
  - Choose the unit of work . . . 8
  - Using tasks in nodes . . . 9
  - `START` node . . . 9
  - `END` node . . . 9
  - Node caching . . . 9
- Edges . . . 9
  - Normal edges . . . 9
  - Conditional edges . . . 9
  - Entry point . . . 10
  - Conditional entry point . . . 10
- `Send` . . . 10
- `Command` . . . 10
  - Return from nodes . . . 11
  - Input to `invoke` or `stream` . . . 11

Correct: plain dict restarts from START **12**
  - Return from tools . . . 12
- Graph migrations . . . 12
- Runtime context . . . 12
  - Recursion limit . . . 13
- Learn more . . . 13

Multi-agent **14**
- Why multi-agent? . . . 14
- Patterns . . . 14
  - Choosing a pattern . . . 14
  - Visual overview . . . 15
- Performance comparison . . . 16

One-shot request 16
Repeat request 18
Multi-domain 19
Summary 20

LangGraph overview

Low-level orchestration framework and runtime for long-running, stateful agents

LangGraph is a low-level orchestration framework and runtime for long-running, stateful agents. You define workflows as graphs of shared state, nodes, and edges, and can mix deterministic steps with LLM-driven steps in the same graph.

LangGraph focuses on orchestration capabilities such as durable execution, streaming, and human-in-the-loop. It does not prescribe prompts or agent architecture.

Before you start, learn the building blocks agents use, especially models and tools. Docs examples often use LangChain for those integrations, but LangGraph does not require LangChain.

If you want a higher-level agent loop first, use Deep Agents or LangChain agents. Use LangGraph when you need explicit control over state and control flow.

How LangChain products fit together

- Deep Agents is an agent harness: planning, subagents, filesystem tools, and context management on top of LangGraph.
- LangChain is the agent framework: abstractions and integrations for models, tools, and agent loops.
- LangGraph is the orchestration runtime: durable execution, streaming, human-in-the-loop, and persistence.
- LangSmith is the platform for tracing, evaluation, prompts, and deployment across frameworks.
- LangSmith Engine detects issues in LangGraph agent traces and proposes fixes. You can open a pull request with the proposed fix from the Engine tab.
- LangSmith Fleet is the no-code agent builder for templates, integrations, and routine automation.

For a comparison of the open source stack, see Frameworks, runtimes, and harnesses.

Install

```
pip install -U langgraph
```

```
uv add langgraph
```

Create a minimal graph:

```
from langgraph.graph import StateGraph, MessagesState, START, END

def mock_llm(state: MessagesState):
    return {"messages": [{"role": "ai", "content": "hello world"}]}

graph = StateGraph(MessagesState)
graph.add_node(mock_llm)
graph.add_edge(START, "mock_llm")
graph.add_edge("mock_llm", END)
graph = graph.compile()

graph.invoke({"messages": [{"role": "user", "content": "hi!"}]})
```

For a full calculator agent, see the LangGraph quickstart.

Capabilities

LangGraph provides the runtime primitives for stateful, long-running workflows. It does not abstract prompts or force a fixed agent architecture.

- **Deterministic and agentic steps**: Combine hand-coded nodes with model-driven nodes in one graph.
- **Persistence**: Checkpoint state and resume after failures or pauses.
- **Human-in-the-loop**: Pause execution, inspect or edit state, then continue.
- **Memory**: Short-term thread state and longer-term memory across sessions.
- **Streaming**: Stream tokens, state updates, and other events as the graph runs.

Integrations, deployment, and observability

LangGraph works on its own. However, you can also pair it with other LangCHain products:

[card] LangChain

Integrations and agent abstractions built on LangGraph.

[card] LangSmith Deployment

Deploy and scale stateful agents, with visual prototyping in Studio.

[card] LangSmith Observability

Trace requests, evaluate outputs, and monitor deployments.

See also

- LangGraph quickstart
- Graph API overview
- Choosing between Graph and Functional APIs
- Frameworks, runtimes, and harnesses

Connect these docs to your agent of choice via MCP for real-time answers.

Edit this page on GitHub or file an issue.

Graph API overview

Learn how LangGraph models agent workflows as graphs of shared state, nodes, and edges.

The Graph API defines agent workflows as a graph of state, nodes, and edges. `Send` and `Command` control how the graph runs work: `Send` dynamically invokes nodes for fan-out, and `Command` updates state while routing to the next node (including resume after an interrupt). Use the Graph API when you want explicit shared state and a visual control flow.

For step-by-step decisions while building, see the Build checklist for the Graph API. For a guided walkthrough, see Thinking in the Graph API. To choose between this and the Functional API, see Choosing between Graph and Functional APIs.

Graphs

LangGraph models agent workflows as graphs built from three parts:

1. `State`: A shared snapshot of the application. Usually a typed schema.
2. `Nodes`: Functions that read state, do work, and return updates.
3. `Edges`: Fixed or conditional transitions that determine the next node.

Compose nodes and edges to build sequences, branches, and loops. Nodes and edges are ordinary functions. They can call an LLM or run other code.

In short: nodes do the work; edges decide what runs next.

Super-steps

LangGraph runs graphs with message passing, in the style of Google's Pregel system. Execution advances in discrete **super-steps**:

- Nodes that run in parallel share one super-step.
- Nodes that run in sequence use separate super-steps.
- A node becomes active when it receives input on an incoming edge.
- After a super-step, nodes with no pending messages become inactive.
- The run ends when every node is inactive and no messages remain in transit.

StateGraph

`StateGraph` is the main graph builder. Parameterize it with your state schema.

Compiling your graph

Build a graph by defining state, adding nodes and edges, then compiling. Compilation checks the graph structure (for example for orphaned nodes). It is also where you attach runtime options such as checkpointers and breakpoints.

You must compile the graph before you invoke or stream it. This is the smallest end-to-end example:

```
from typing_extensions import TypedDict

from langgraph.graph import END, START, StateGraph


class State(TypedDict):
    text: str


def node_a(state: State):
    return {"text": state["text"] + "a"}


graph = (
```

```
    StateGraph(State)
    .add_node("node_a", node_a)
    .add_edge(START, "node_a")
    .add_edge("node_a", END)
    .compile()
)

graph.invoke({"text": ""})
# {'text': 'a'}
```

State

State is the shared data that nodes read and update. LangGraph stores that data in **channels**: named slots in the state schema (the keys of your state object). Each channel holds a value and defines how updates merge into it. Most channels use last-write-wins. Channels with a reducer combine values instead, for example by appending to a list.

State has two parts:

- A schema that declares the channels in the graph
- Optional reducers that control how updates merge into each channel

Nodes return partial updates. LangGraph applies those updates through each channel's reducer.

Define state with a `TypedDict`, a `dataclass` (for defaults), or a Pydantic `BaseModel` when you need validation. Prefer `TypedDict` or `dataclass` for performance.

> [!NOTE]
>
> `create_agent` state schemas must be `TypedDict`. Pydantic models are not supported.

Schema

By default, the graph input and output schemas match the full state. You can narrow them when some keys are internal only. See Define input and output schemas.

Multiple schemas Most graphs share one state schema. Use multiple schemas when you need either of the following:

- Private channels that internal nodes share, but that are not part of graph input or output
- Explicit input and output schemas that are subsets of the full internal state

A node can write to any channel in the graph state, including channels outside its declared input schema. Nodes can also introduce private channels when those schemas are defined. Input and output schemas constrain what `invoke` accepts and returns. They do not hide channels from streaming by default. See Private channels and streaming.

For examples, see Define input and output schemas and Pass private state between nodes.

Reducers

Reducers control how node updates merge into state. Each key has its own reducer. With no reducer, updates overwrite the previous value.

Reducer arguments A reducer is a binary function:

- **Left**: The current value in state for that key
- **Right**: The update returned by a node for that key

```
new_value = reducer(left=current_state[key], right=node_update[key])
```

Define a reducer and attach it to a state key with `Annotated`:

```
from typing import Annotated
```

```python
from typing_extensions import TypedDict


def append_strings(left: list[str], right: list[str]) -> list[str]:
    """Combine the existing state value (left) with a node update (right)."""
    return left + right


class State(TypedDict):
    tags: Annotated[list[str], append_strings]
```

Calling the reducer with left and right shows how values combine:

```python
append_strings(left=["draft"], right=["review"])  # returns ["draft", "review"]
```

Default reducer The default reducer replaces the left value with the right. Nodes return partial updates, not the full state.

For example, start with `{"foo": 1, "bar": ["hi"]}`. An update of `{"foo": 2}` yields `{"foo": 2, "bar": ["hi"]}`. A later `{"bar": ["bye"]}` yields `{"foo": 2, "bar": ["bye"]}`.

Custom reducers Use a custom reducer to combine values, such as appending to a list.

```python
import operator
from typing import Annotated

from typing_extensions import TypedDict


class State(TypedDict):
    foo: int
    bar: Annotated[list[str], operator.add]
```

Here `foo` overwrites. `bar` appends with `operator.add`. From `{"foo": 1, "bar": ["hi"]}`, an update of `{"bar": ["bye"]}` becomes `{"foo": 1, "bar": ["hi", "bye"]}`.

Overwrite To bypass a merging reducer and replace a value (including clearing it), use `Overwrite`. See Bypass reducers with Overwrite.

Resetting a reducer field With a merging reducer, returning an empty value keeps prior data. That matters for error buffers or retry counters that must clear between attempts. See Resetting a reducer field and Bypass reducers with Overwrite.

Working with messages in graph state

Why use messages? Chat models take a list of messages as input. LangChain's chat model interface accepts message objects such as `HumanMessage` and `AIMessage`. See the Messages conceptual guide.

Using messages in your graph Store conversation history as a list of messages in state, with a reducer that appends new messages and updates existing ones by ID.

Use `add_messages` on a `messages` key. Plain `operator.add` only appends, so it cannot update an existing message in place during human-in-the-loop edits.

```python
from typing import Annotated

from langchain.messages import AnyMessage
from langgraph.graph.message import add_messages
from typing_extensions import TypedDict
```

```python
class GraphState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
```

Serialization Message reducers also deserialize common wire formats into LangChain message objects.

```python
# Supported
{"messages": [HumanMessage(content="message")]}

# Also supported
{"messages": [{"type": "human", "content": "message"}]}
```

Access attributes with dot notation, for example `state["messages"][-1].content`. See LangChain serialization.

MessagesState `MessagesState` is a prebuilt state with a single `messages` key that uses `add_messages`. Subclass it to add fields:

```python
from langgraph.graph import MessagesState


class State(MessagesState):
    documents: list[str]
```

Nodes

Nodes are sync or async functions that implement a step of the graph.

A node can accept:

1. `state`: The graph state
2. `config`: A `RunnableConfig` with values such as `thread_id` and `tags`
3. `runtime`: A `Runtime` object with runtime context, `store`, `stream_writer`, `execution_info`, and related helpers

Add nodes with `add_node`:

```python
from dataclasses import dataclass

from langgraph.graph import StateGraph
from langgraph.runtime import Runtime
from typing_extensions import TypedDict


class State(TypedDict):
    input: str
    results: str


@dataclass
class Context:
    user_id: str


builder = StateGraph(State)


def plain_node(state: State):
    return state
```

```python
def node_with_runtime(state: State, runtime: Runtime[Context]):
    return {"results": f"Hello, {state['input']}!"}


builder.add_node("plain_node", plain_node)
builder.add_node("node_with_runtime", node_with_runtime)
```

Behind the scenes, LangGraph wraps functions as `RunnableLambda` so they gain batching, async support, and tracing.

If you omit a node name, LangGraph uses the function name.

```python
builder.add_node(my_node)
# Reference the node as "my_node" in edges
```

Re-execution and idempotency

With a checkpointer, LangGraph saves checkpoints at super-step boundaries, not mid-function inside a node. After an interrupt or retry, the affected node runs again from the start of its function. Side effects before the pause run again.

Design node logic so re-execution is safe. Prefer idempotency keys, upserts, or read-before-write checks. For durable keys and external side effects under retries, see Retry boundaries and idempotency. For effects around `interrupt()`, see Side effects called before `interrupt` must be idempotent.

You can add or remove nodes and edges without breaking resume for existing threads. Resumed runs use saved state and the graph you compile now.

If a node calls tasks or `interrupt`, stricter determinism rules apply on resume. Completed task results are restored from the checkpointer. Changing task or interrupt order before the resume point can mismatch cached values. See Idempotency.

Choose the unit of work

A node is a checkpoint and retry boundary. It is also where the graph can take a different path. Deterministic work such as parsing, retrieval, or a fixed tool chain does not need its own node unless you want that boundary.

Ask whether the step decides what happens next, or executes something already decided. Then pick the primitive that matches the recovery behavior you need:

Primitive	Own checkpoint	On resume or retry	Choose when
Plain function inside a node	No. It runs inside the node's super-step.	Re-runs whenever the node re-runs.	No routing fork after the step, and repeating it with the rest of the node is acceptable.
Task	No new graph checkpoint. The result is stored with the running entrypoint or parent checkpoint.	The enclosing entrypoint or node replays from the start. Completed tasks return stored results.	The step has a side effect or is expensive, and must not repeat on resume.
Subgraph	Yes for stateful modes: nodes checkpoint under a namespace of the parent thread. Compile with `checkpointer=False` for no subgraph checkpoints.	Stateful modes resume inside the subgraph at the interrupted or failed node.	The unit has its own routing or state schema, or more than one parent graph reuses it.
Graph node	Yes. A checkpoint follows the super-step it runs in.	Resume starts at that node. Earlier nodes do not re-run.	A routing fork is possible after the step, or you need an interrupt, resume, or time travel boundary.

For side effects under retries, see Retry boundaries and idempotency. For task retries and timeouts, see Tasks.

Using tasks in nodes

When one node runs several operations that should checkpoint independently, wrap each operation in a task instead of splitting the work across nodes. With a checkpointer, completed task results are saved, so a resumed thread can skip finished work inside the node. For patterns and examples, see Tasks and Common pitfalls.

`START` node

`START` is the virtual entry that delivers user input into the graph. Use it to mark which nodes run first.

```
from langgraph.graph import START

graph.add_edge(START, "node_a")
```

`END` node

`END` marks a terminal edge. Use it when a path should stop.

```
from langgraph.graph import END

graph.add_edge("node_a", END)
```

Node caching

LangGraph can cache node (and task) results by input. Pass a cache when you compile the graph, and attach a `CachePolicy` on the node.

For options, end-to-end examples, and graph-wide defaults, see Add node caching.

Edges

Edges decide where control goes next:

- **Normal edges**: Always go from one node to the next
- **Conditional edges**: Call a function to determine the next node(s)
- **Entry point**: First node after user input (`START` -> node)
- **Conditional entry point**: Route from `START` with a function

A node can have multiple outgoing edges. Destinations on those edges run in parallel in the next super-step.

> [!WARNING]
>
> For each node, use either static edges or dynamic routing (conditional edges / `Command`), not both. Mixing them can run both paths.

Normal edges

If you always want to go from node A to node B, use `add_edge`:

```
graph.add_edge("node_a", "node_b")
```

Conditional edges

If you want to optionally route to one or more nodes (or terminate), use `add_conditional_edges`. Pass the source node and a routing function to call after that node runs:

```
graph.add_conditional_edges("node_a", routing_function)
# Or with an explicit map:
graph.add_conditional_edges(
    "node_a",
    routing_function,
```

```
    {"path_a": "node_b", "path_b": "node_c"},
)
```

The routing function receives the current graph state. By default, its return value is the name of the next node (or a list of names). Those nodes run in parallel in the next super-step. You can also pass a dictionary that maps the function's output to node names.

The routing function can return a node name, `END`, a list of names (fan-out), or `Send` objects for map-reduce. See Build sequences, branches, and loops.

> [!TIP]
>
> Use `Command` instead of conditional edges when you want to combine state updates and routing in a single function.

Entry point

The entry point is the first node (or nodes) that run when the graph starts. Add an edge from the virtual `START` node to the first node to execute:

```
graph.add_edge(START, "node_a")
```

Conditional entry point

A conditional entry point lets you start at different nodes depending on custom logic. Use `add_conditional_edges` from `START`:

```
graph.add_conditional_edges(START, routing_function)
```

You can optionally provide a dictionary that maps the routing function's output to the next node name.

Send

Use `Send` when you need to run the same node once per item, each with its own input, and the number of items is only known at runtime. That shape is often called map-reduce or orchestrator-worker: one node produces a list, workers process each item in parallel, then a later node reads the combined results.

Normal edges cannot do this. They choose which nodes run next, but every chosen node still receives the shared graph state, and the topology is fixed when you build the graph.

Return one or more `Send` objects from a conditional edge. Each `Send` names a target node and passes a custom argument as that invocation's input. LangGraph runs those invocations in parallel. When workers write to shared state keys, reducers merge their updates, for example by concatenating lists.

The first argument to `Send` is the target node name. The second is the state (or partial state) for that invocation.

```
from langgraph.types import Send


def continue_to_jokes(state: OverallState):
    return [Send("generate_joke", {"subject": s}) for s in state["subjects"]]


graph.add_conditional_edges("node_a", continue_to_jokes)
```

For a full map-reduce example, see Use Send for map-reduce.

Command

Use `Command` when a single step must both update state and choose what runs next. Normally those jobs are separate: a node returns state updates, and edges (or conditional edges) decide the next node. Return `Command` from a node when the next hop depends on work you just did in that same function.

`Command` also covers two other cases that plain returns and edges do not: resume a run after an interrupt, and update or route from inside a tool.

For routing alone, prefer conditional edges. Reach for `Command` when you need the update and the route together (or one of the interrupt / tool cases above).

`Command` accepts:

- **`update`**: State updates (like a normal node return)
- **`goto`**: Next node(s) (like a conditional edge)
- **`graph`**: Parent graph target when navigating from a subgraph
- **`resume`**: Value used to resume after an interrupt

Use `Command` in three places:

- **Return from nodes**: `update`, `goto`, and `graph`
- **Input to `invoke` or `stream`**: `resume` after an interrupt
- **Return from tools**: Updates and routing from inside a tool

Return from nodes

`update` and `goto` Return `Command` to update state and route in one step:

```python
def my_node(state: CommandState) -> Command[Literal["my_other_node"]]:
    return Command(
        update={"foo": "bar"},
        goto="my_other_node",
    )
```

Annotate the return type with the allowed destinations, for example `Command[Literal["my_other_node"]]`. That tells LangGraph which edges to render.

Use `Command` when you need both an update and a route. For routing alone, prefer conditional edges.

> [!WARNING]
>
> `Command` adds dynamic edges. Static edges from the same node still run. Do not mix `Command` routing with `add_edge` from one node.

See Combine control flow and state updates with Command.

`graph` From a subgraph node, navigate to a node in the parent graph with `Command.PARENT`:

```python
def my_node(state: CommandState) -> Command[Literal["other_subgraph"]]:
    return Command(
        update={"foo": "bar"},
        goto="other_subgraph",  # where `other_subgraph` is a node in the parent graph
        graph=Command.PARENT,
    )
```

When you update a parent key that the subgraph also uses, define a reducer for that key in the parent. See Navigate to a node in a parent graph.

Input to `invoke` or `stream`

> [!WARNING]
>
> `Command(resume=...)` is the only `Command` pattern meant as input to `invoke()` / `stream()` (optionally with `update=...` while resuming). Do not pass `Command(update=...)` alone to continue a conversation. Any `Command` as input resumes from the latest checkpoint, not from `START`, so a finished thread can look stuck. Pass a plain dict to start a new turn:
>
> “‘python theme={“theme”:{“light”:“catppuccin-latte”,“dark”:“catppuccin-mocha”}} # Wrong: resumes from the latest checkpoint graph.invoke(Command(update={ # [!code –] “messages”: [{“role”: “user”, “content”: “follow up”}] # [!code –] }), config) # [!code –]

Correct: plain dict restarts from START

graph.invoke({ # [!code ++] “messages”: [{“role”: “user”, “content”: “follow up”}] # [!code ++] }, config) # [!code ++] “‘

resume Use `Command(resume=...)` after an interrupt. The value becomes the return value of the paused `interrupt()` call:

```
graph.invoke(Command(resume="yes"), config)
```

See Interrupts for full patterns.

Return from tools

Return `Command` from tools to update state and route after the tool finishes. `goto` from a tool adds a dynamic edge. Static edges on the calling node still run, so do not mix both.

See Use inside tools.

Graph migrations

You can change a graph’s nodes, edges, and state schema even when a checkpointer already stores thread state. What is safe depends on whether the thread has finished or is paused.

Topology (nodes and edges)

- Finished threads (not interrupted): you can add, remove, or rename nodes and edges freely.
- Interrupted threads: you can change edges and add nodes. Do not rename or remove a node the thread is about to enter. On resume, LangGraph looks up that node by name, and a missing name fails the run.

State schema

- Adding or removing keys is backwards and forwards compatible.
- Renaming a key drops the saved value for that key in existing threads.
- Incompatible type changes on an existing key (for example, narrowing or requiring a field that old checkpoints omit) can break older thread state.

For logic changes that stay schema-compatible (for example, a new tool set), see Business compatibility.

Runtime context

Runtime context passes dependencies into nodes without putting them in graph state (for example, model name or a database handle).

```
from dataclasses import dataclass

from langgraph.graph import END, START, StateGraph
from langgraph.runtime import Runtime
from typing_extensions import TypedDict


class State(TypedDict):
    input: str
    output: str


@dataclass
class ContextSchema:
    llm_provider: str = "openai"


def get_llm(provider: str) -> str:
```

```
    return provider


def node_a(state: State, runtime: Runtime[ContextSchema]):
    llm = get_llm(runtime.context.llm_provider)
    return {"output": llm}


graph = (
    StateGraph(State, context_schema=ContextSchema)
    .add_node("node_a", node_a)
    .add_edge(START, "node_a")
    .add_edge("node_a", END)
    .compile()
)
graph.invoke({"input": "hi"}, context={"llm_provider": "anthropic"})
```

See Add runtime configuration.

Recursion limit

The recursion limit caps how many super-steps one run may execute. When the limit is exceeded, LangGraph raises `GraphRecursionError`.

Starting in version 1.1.4, the default is 10,007 super-steps. Between 1.0.6 and 1.1.3, the default is 10,000 steps. Override it with the `LANGGRAPH_DEFAULT_RECURSION_LIMIT` environment variable, or pass `recursion_limit` as a top-level config key (not under `configurable`).

For recipes (catching the error, reading `langgraph_step`, and Python `RemainingSteps`), see Impose a recursion limit and Access and handle the recursion counter.

Learn more

- Thinking in the Graph API
- LangGraph quickstart
- Build checklist for the Graph API
- Define and update state
- Build sequences, branches, and loops
- Use Send and Command
- Configure nodes
- Visualize your graph
- Functional API overview

Connect these docs to your agent of choice via MCP for real-time answers.

Edit this page on GitHub or file an issue.

Multi-agent

Multi-agent systems coordinate specialized components to tackle complex workflows. However, not every complex task requires this approach–a single agent with the right (sometimes dynamic) tools and prompt can often achieve similar results.

> [!TIP]
>
> For built-in multi-agent support, use Managed Deep Agents: a managed higher-level harness built on LangChain that ships with subagents, skills, planning, a virtual filesystem, and context management. Alternatively, for maximum flexibility while using the same Deep Agents harness, use Deep Agents and host agents yourself, or run a custom LangSmith Deployment.

Why multi-agent?

When developers say they need "multi-agent," they're usually looking for one or more of these capabilities:

- **Context management**: Provide specialized knowledge without overwhelming the model's context window. If context were infinite and latency zero, you could dump all knowledge into a single prompt–but since it's not, you need patterns to selectively surface relevant information.
- **Distributed development**: Allow different teams to develop and maintain capabilities independently, composing them into a larger system with clear boundaries.
- **Parallelization**: Spawn specialized workers for subtasks and execute them concurrently for faster results.

Multi-agent patterns are particularly valuable when a single agent has too many tools and makes poor decisions about which to use, when tasks require specialized knowledge with extensive context (long prompts and domain-specific tools), or when you need to enforce sequential constraints that unlock capabilities only after certain conditions are met.

> [!TIP]
>
> At the center of multi-agent design is **context engineering**–deciding what information each agent sees. The quality of your system depends on ensuring each agent has access to the right data for its task.

Patterns

Here are the main patterns for building multi-agent systems, each suited to different use cases:

Pattern	How it works
Subagents	A main agent coordinates subagents as tools. All routing passes through the main agent, which decides when and how to invoke each subagent.
Handoffs	Behavior changes dynamically based on state. Tool calls update a state variable that triggers routing or configuration changes, switching agents or adjusting the current agent's tools and prompt.
Skills	Specialized prompts and knowledge loaded on-demand. A single agent stays in control while loading context from skills as needed.
Router	A routing step classifies input and directs it to one or more specialized agents. Results are synthesized into a combined response.
Custom workflow	Build bespoke execution flows with LangGraph, mixing deterministic logic and agentic behavior. Embed other patterns as nodes in your workflow.

Choosing a pattern

Use this table to match your requirements to the right pattern:

Pattern	Distributed development	Parallelization	Multi-hop	Direct user interaction
Subagents				
Handoffs	-	-		
Skills				
Router			-	

- **Distributed development**: Can different teams maintain components independently?
- **Parallelization**: Can multiple agents execute concurrently?
- **Multi-hop**: Does the pattern support calling multiple subagents in series?
- **Direct user interaction**: Can subagents converse directly with the user?

[!TIP]

You can mix patterns! For example, a **subagents** architecture can invoke tools that invoke custom workflows or router agents. Subagents can even use the **skills** pattern to load context on-demand. The possibilities are endless!

Visual overview

[tab] Subagents

A main agent coordinates subagents as tools. All routing passes through the main agent.

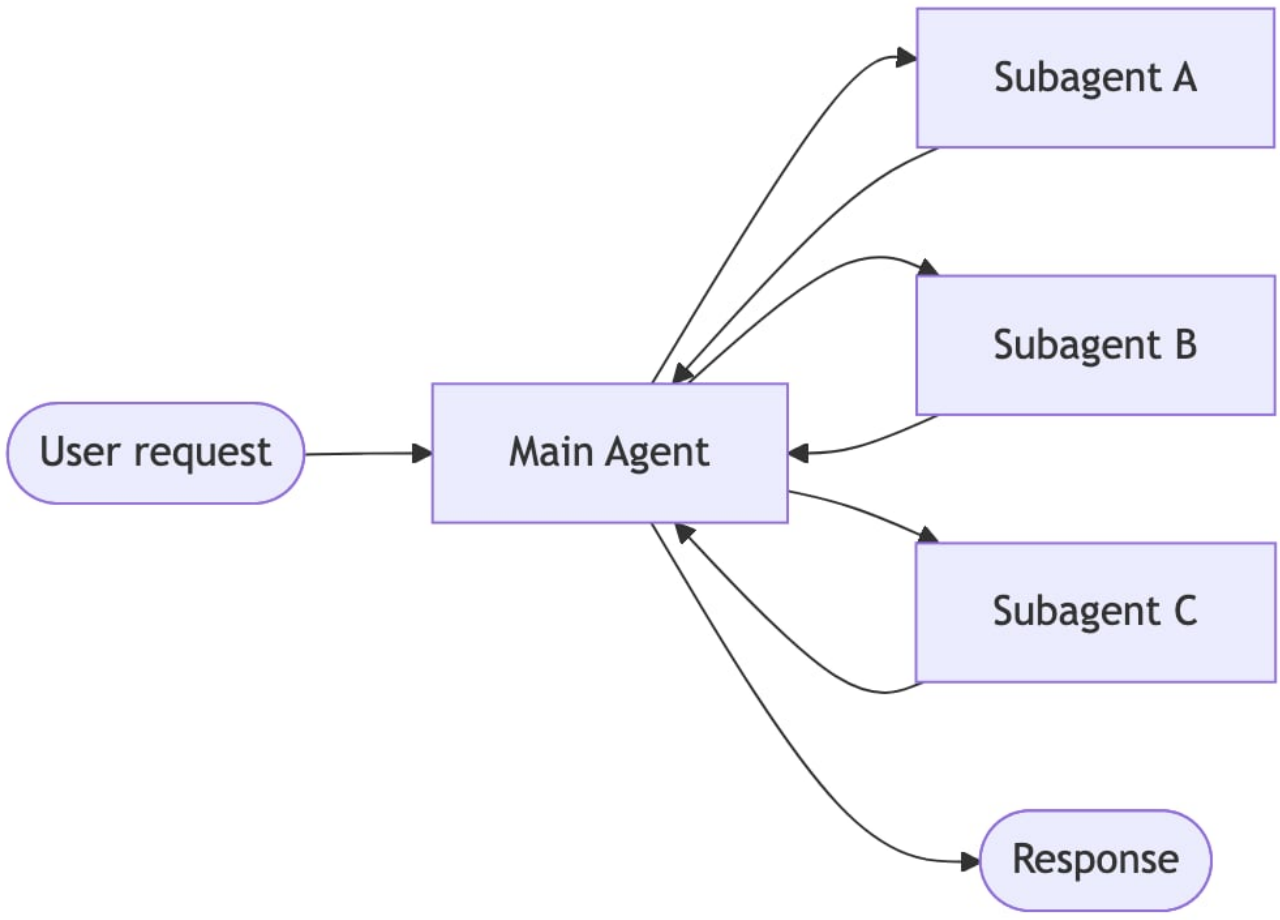


Figure 1: Subagents pattern: main agent coordinates subagents as tools

[tab] Handoffs

Agents transfer control to each other via tool calls. Each agent can hand off to others or respond directly to the user.

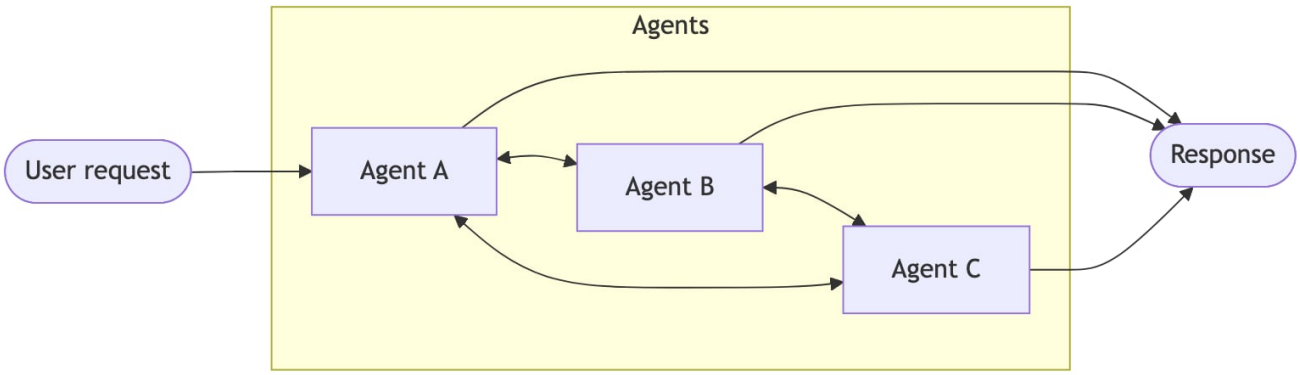


Figure 2: Handoffs pattern: agents transfer control via tool calls

[tab] Skills

A single agent loads specialized prompts and knowledge on-demand while staying in control.

[tab] Router

A routing step classifies input and directs it to specialized agents. Results are synthesized.

[!TIP]

Trace the full coordination flow across agents with LangSmith. Follow the tracing quickstart to get set up.

We recommend you also set up LangSmith Engine which monitors your traces, detects issues, and proposes fixes.

Performance comparison

Different patterns have different performance characteristics. Understanding these tradeoffs helps you choose the right pattern for your latency and cost requirements.

Key metrics:

- **Model calls**: Number of LLM invocations. More calls = higher latency (especially if sequential) and higher per-request API costs.
- **Tokens processed**: Total context window usage across all calls. More tokens = higher processing costs and potential context limits.

One-shot request

User: "Buy coffee"

A specialized coffee agent/skill can call a `buy_coffee` tool.

Pattern	Model calls	Best fit
Subagents	4	
Handoffs	3	
Skills	3	
Router	3	

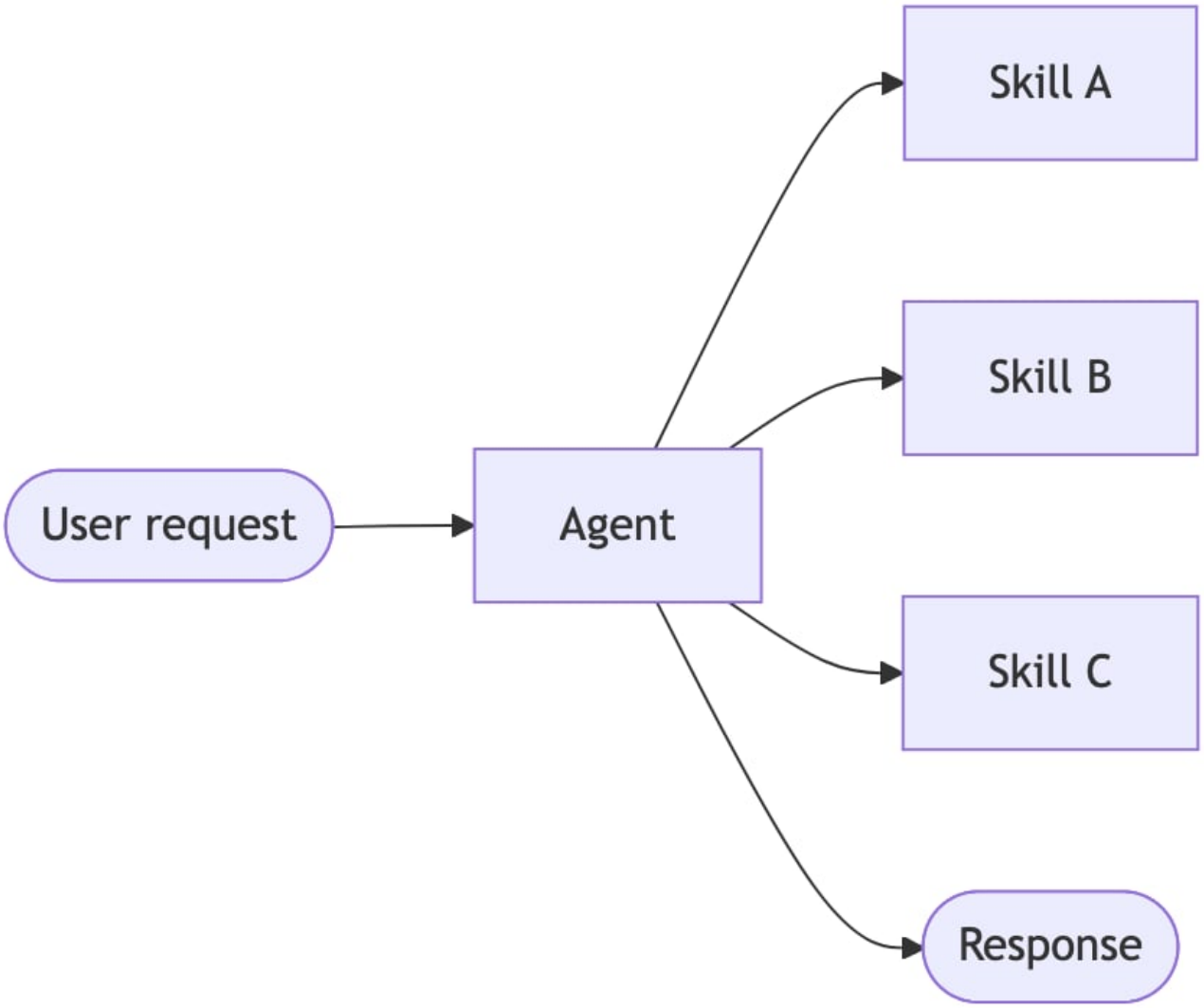


Figure 3: Skills pattern: single agent loads specialized context on-demand

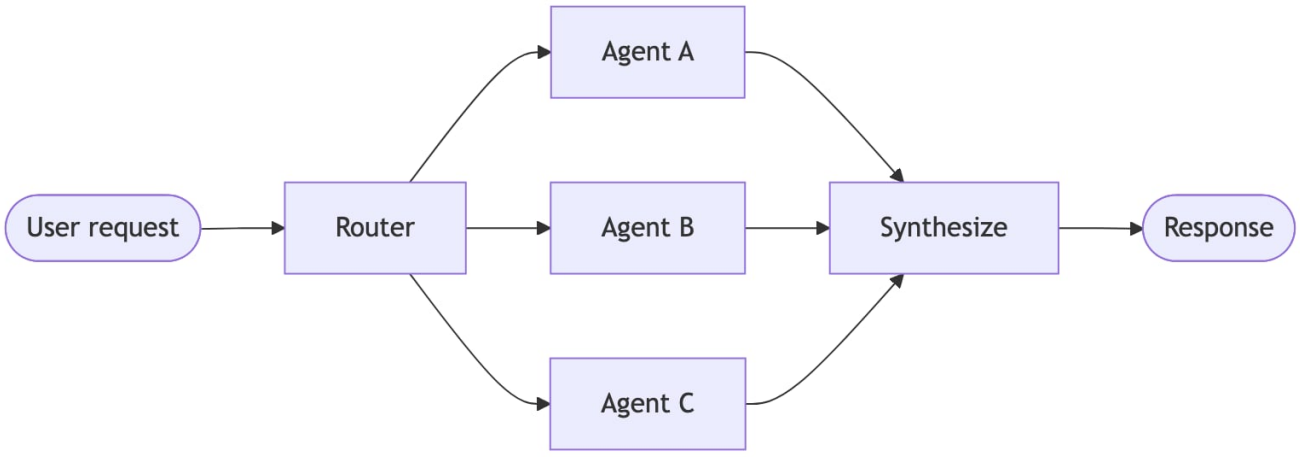


Figure 4: Router pattern: routing step classifies input to specialized agents

[tab] Subagents

4 model calls:

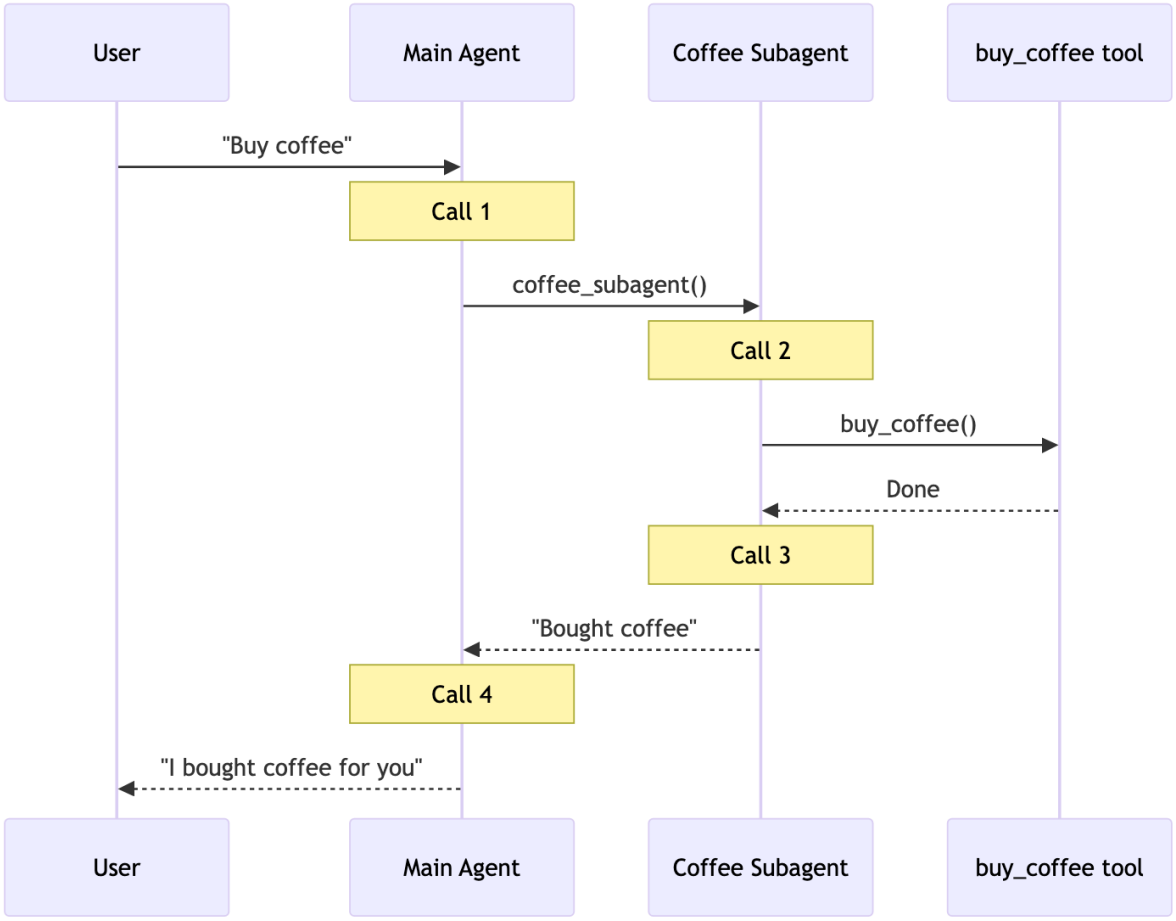


Figure 5: Subagents one-shot: 4 model calls for buy coffee request

[tab] Handoffs

3 model calls:

[tab] Skills

3 model calls:

[tab] Router

3 model calls:

Key insight: Handoffs, Skills, and Router are most efficient for single tasks (3 calls each). Subagents adds one extra call because results flow back through the main agent–this overhead provides centralized control.

Repeat request

Turn 1: "Buy coffee" **Turn 2:** "Buy coffee again"

The user repeats the same request in the same conversation.

Pattern	Turn 2 calls	Total (both turns)	Best fit
Subagents	4	8	

Pattern	Turn 2 calls	Total (both turns)	Best fit
Handoffs	2	5	
Skills	2	5	
Router	3	6	

[tab] Subagents

4 calls again -> 8 total

- Subagents are **stateless by design**–each invocation follows the same flow
- The main agent maintains conversation context, but subagents start fresh each time
- This provides strong context isolation but repeats the full flow

[tab] Handoffs

2 calls -> 5 total

- The coffee agent is **still active** from turn 1 (state persists)
- No handoff needed–agent directly calls `buy_coffee` tool (call 1)
- Agent responds to user (call 2)
- **Saves 1 call by skipping the handoff**

[tab] Skills

2 calls -> 5 total

- The skill context is **already loaded** in conversation history
- No need to reload–agent directly calls `buy_coffee` tool (call 1)
- Agent responds to user (call 2)
- **Saves 1 call by reusing loaded skill**

[tab] Router

3 calls again -> 6 total

- Routers are **stateless**–each request requires an LLM routing call
- Turn 2: Router LLM call (1) -> Milk agent calls buy_coffee (2) -> Milk agent responds (3)
- Can be optimized by wrapping as a tool in a stateful agent

Key insight: Stateful patterns (Handoffs, Skills) save 40-50% of calls on repeat requests. Subagents maintain consistent cost per request–this stateless design provides strong context isolation but at the cost of repeated model calls.

Multi-domain

User: "Compare Python, JavaScript, and Rust for web development"

Each language agent/skill contains ~2000 tokens of documentation. All patterns can make parallel tool calls.

Pattern	Model calls	Total tokens	Best fit
Subagents	5	~9K	
Handoffs	7+	~14K+	
Skills	3	~15K	
Router	5	~9K	

[tab] Subagents

5 calls, ~9K tokens

Each subagent works in **isolation** with only its relevant context. Total: **9K tokens**.

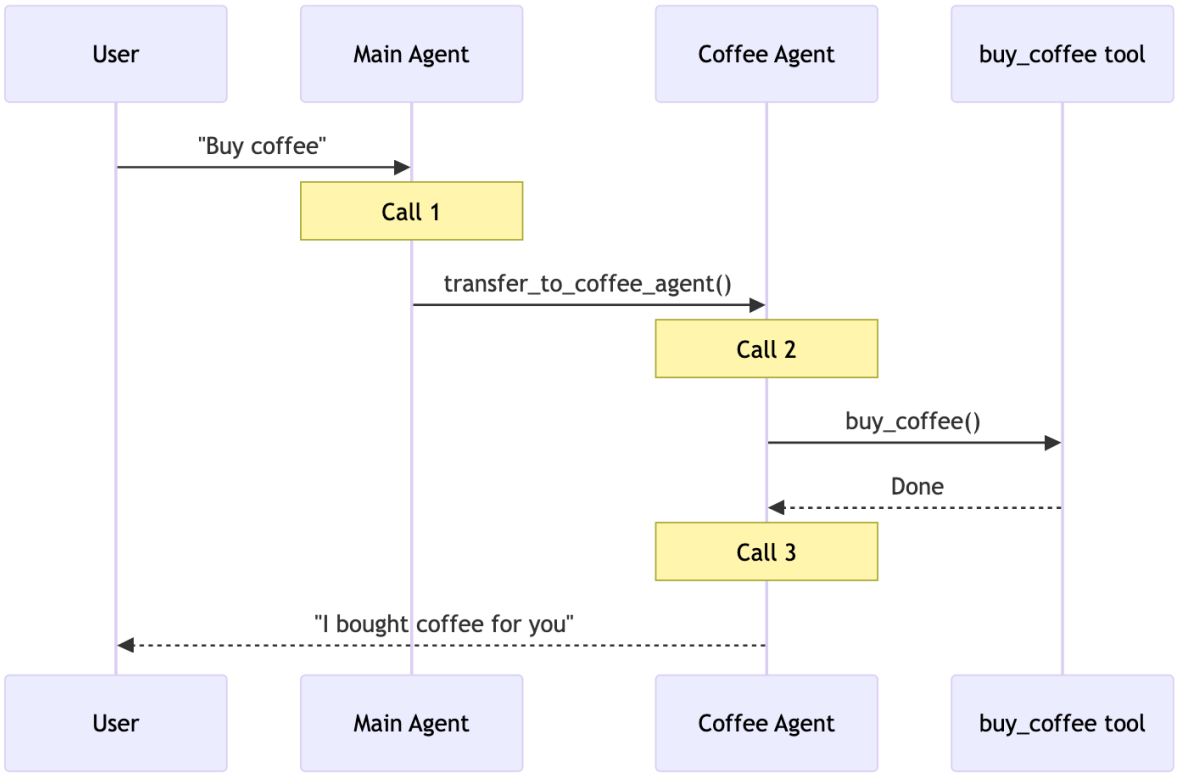


Figure 6: Handoffs one-shot: 3 model calls for buy coffee request

[tab] Handoffs

7+ calls, ~14K+ tokens

Handoffs executes **sequentially**–can't research all three languages in parallel. Growing conversation history adds overhead. Total: **~14K+ tokens**.

[tab] Skills

3 calls, ~15K tokens

After loading, **every subsequent call processes all 6K tokens of skill documentation**. Subagents processes 67% fewer tokens overall due to context isolation. Total: **15K tokens**.

[tab] Router

5 calls, ~9K tokens

Router uses an **LLM for routing**, then invokes agents in parallel. Similar to Subagents but with explicit routing step. Total: **9K tokens**.

Key insight: For multi-domain tasks, patterns with parallel execution (Subagents, Router) are most efficient. Skills has fewer calls but high token usage due to context accumulation. Handoffs is inefficient here–it must execute sequentially and can't leverage parallel tool calling for consulting multiple domains simultaneously.

Summary

Here's how patterns compare across all three scenarios:

Pattern	One-shot	Repeat request	Multi-domain
Subagents	4 calls	8 calls (4+4)	5 calls, 9K tokens
Handoffs	3 calls	5 calls (3+2)	7+ calls, 14K+ tokens

Pattern	One-shot	Repeat request	Multi-domain
Skills	3 calls	5 calls (3+2)	3 calls, 15K tokens
Router	3 calls	6 calls (3+3)	5 calls, 9K tokens

Choosing a pattern:

Optimize for	Subagents	Handoffs	Skills	Router
Single requests				
Repeat requests				
Parallel execution				
Large-context domains				
Simple, focused tasks				

Connect these docs to your agent of choice via MCP for real-time answers.

Edit this page on GitHub or file an issue.

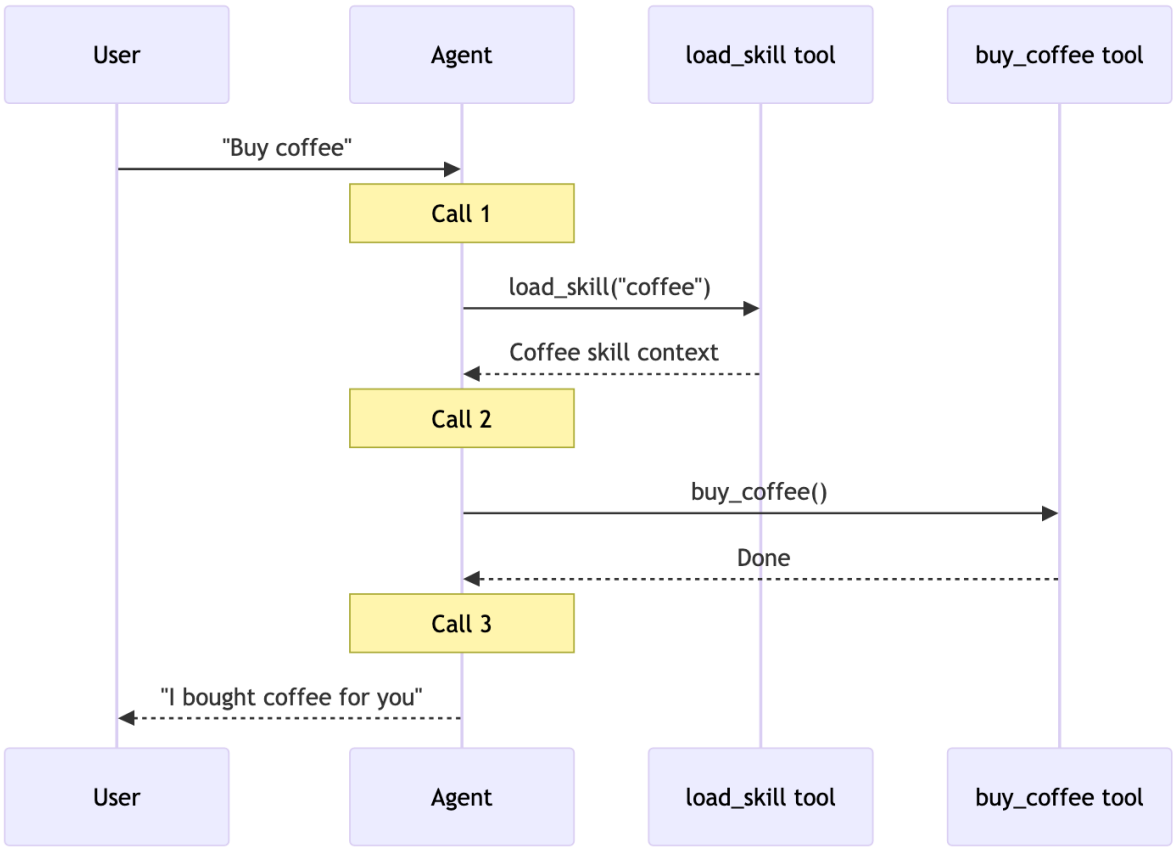


Figure 7: Skills one-shot: 3 model calls for buy coffee request

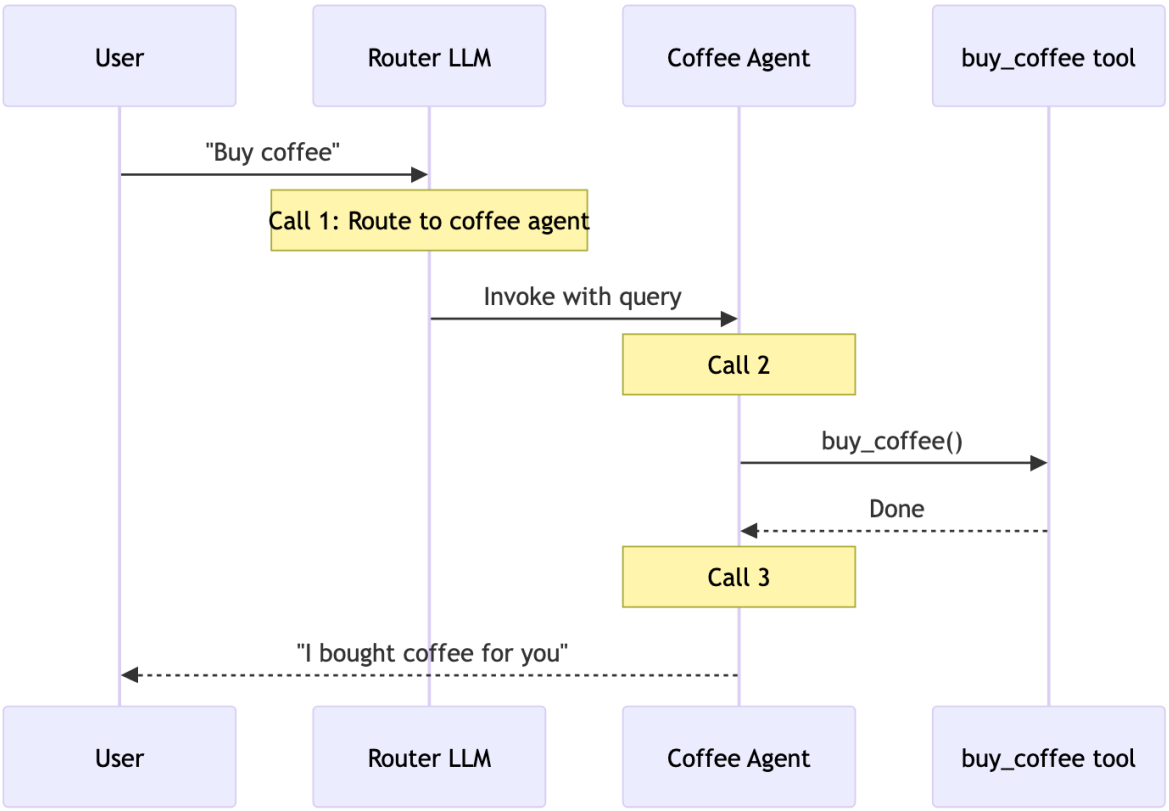


Figure 8: Router one-shot: 3 model calls for buy coffee request

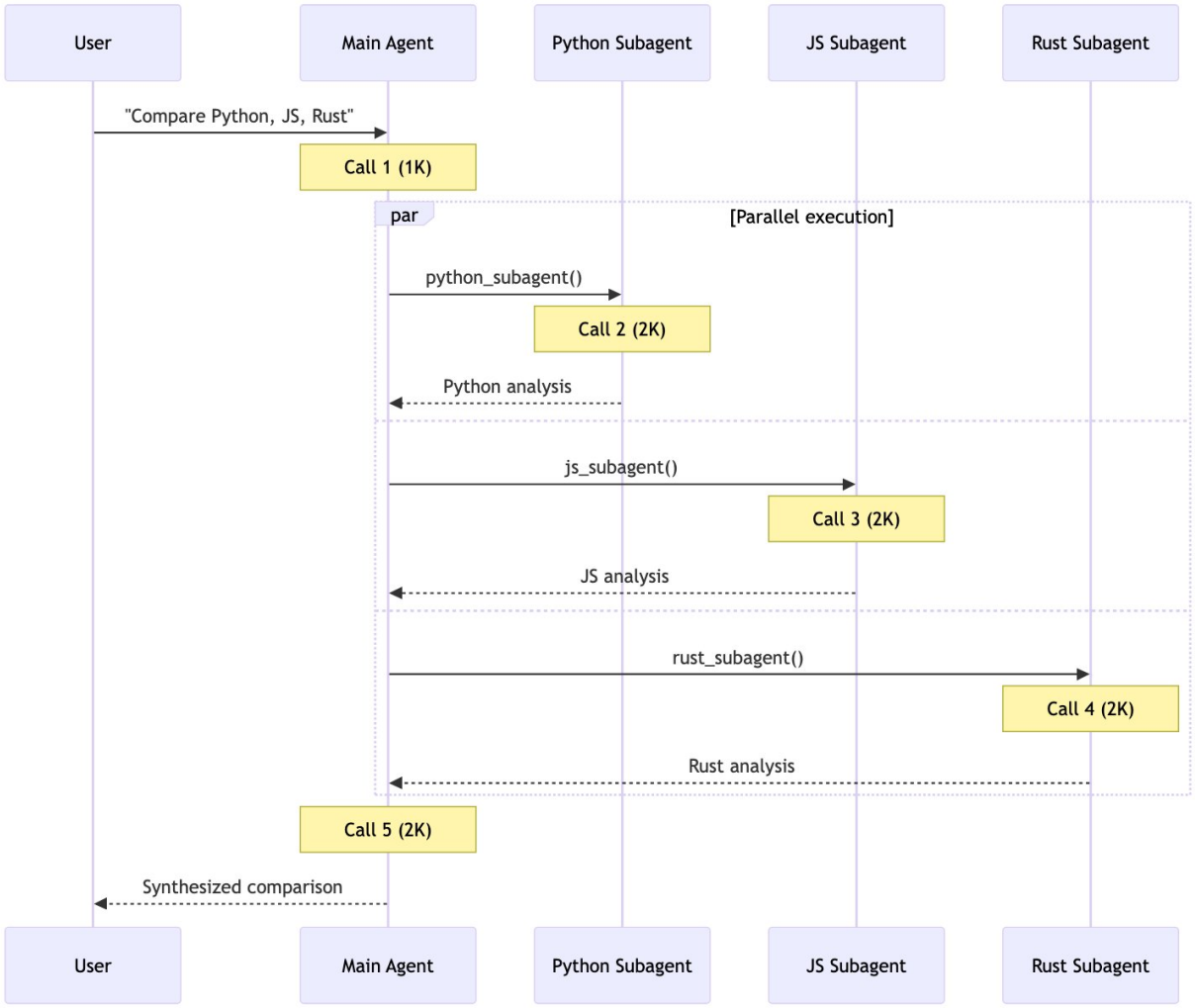


Figure 9: Subagents multi-domain: 5 calls with parallel execution

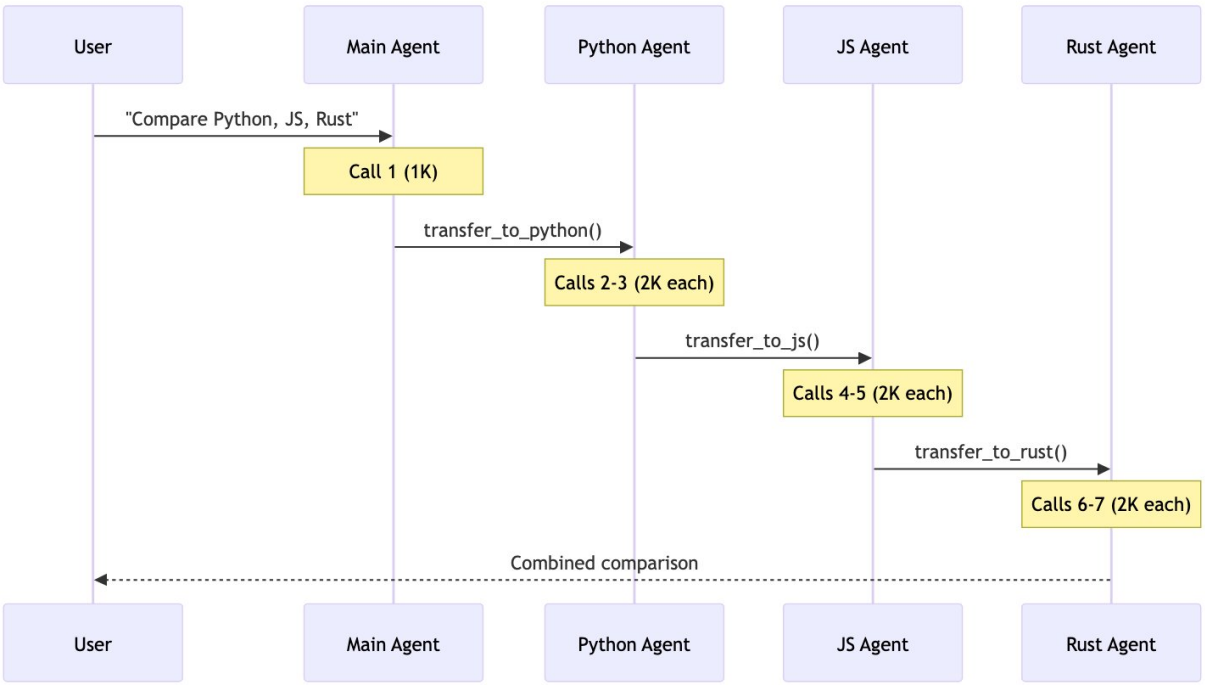


Figure 10: Handoffs multi-domain: 7+ sequential calls

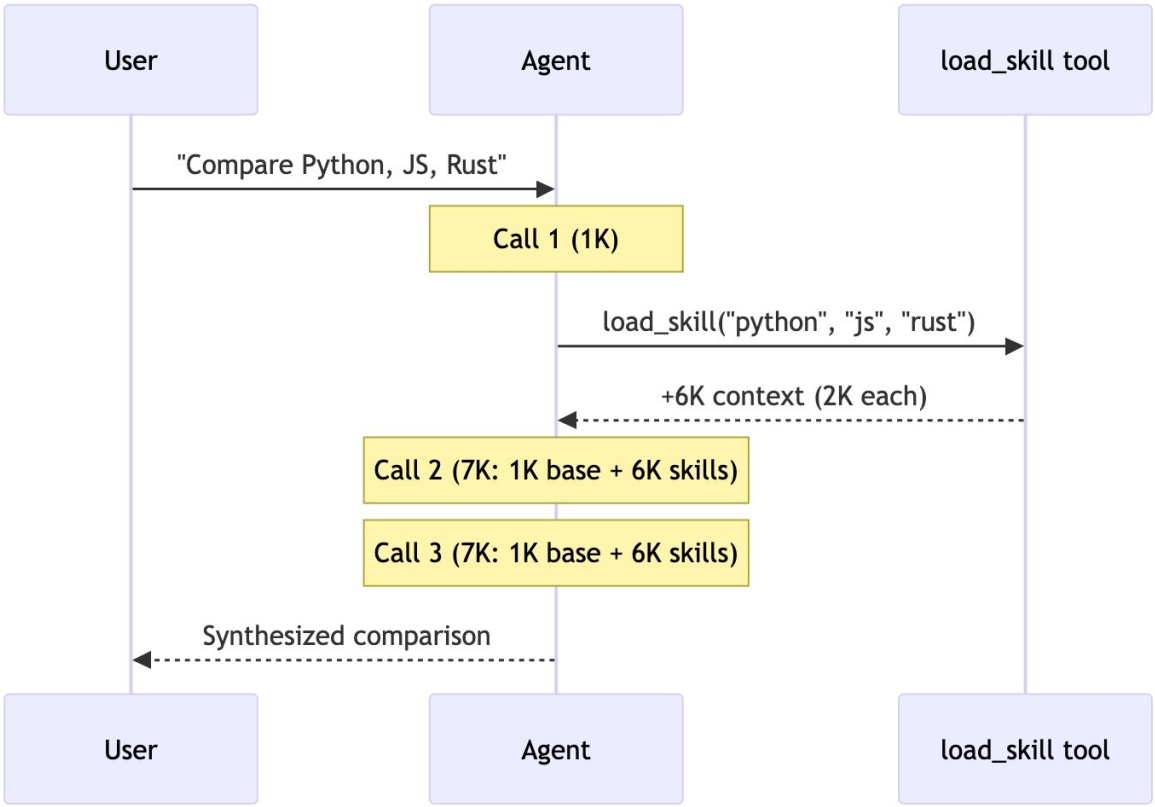


Figure 11: Skills multi-domain: 3 calls with accumulated context

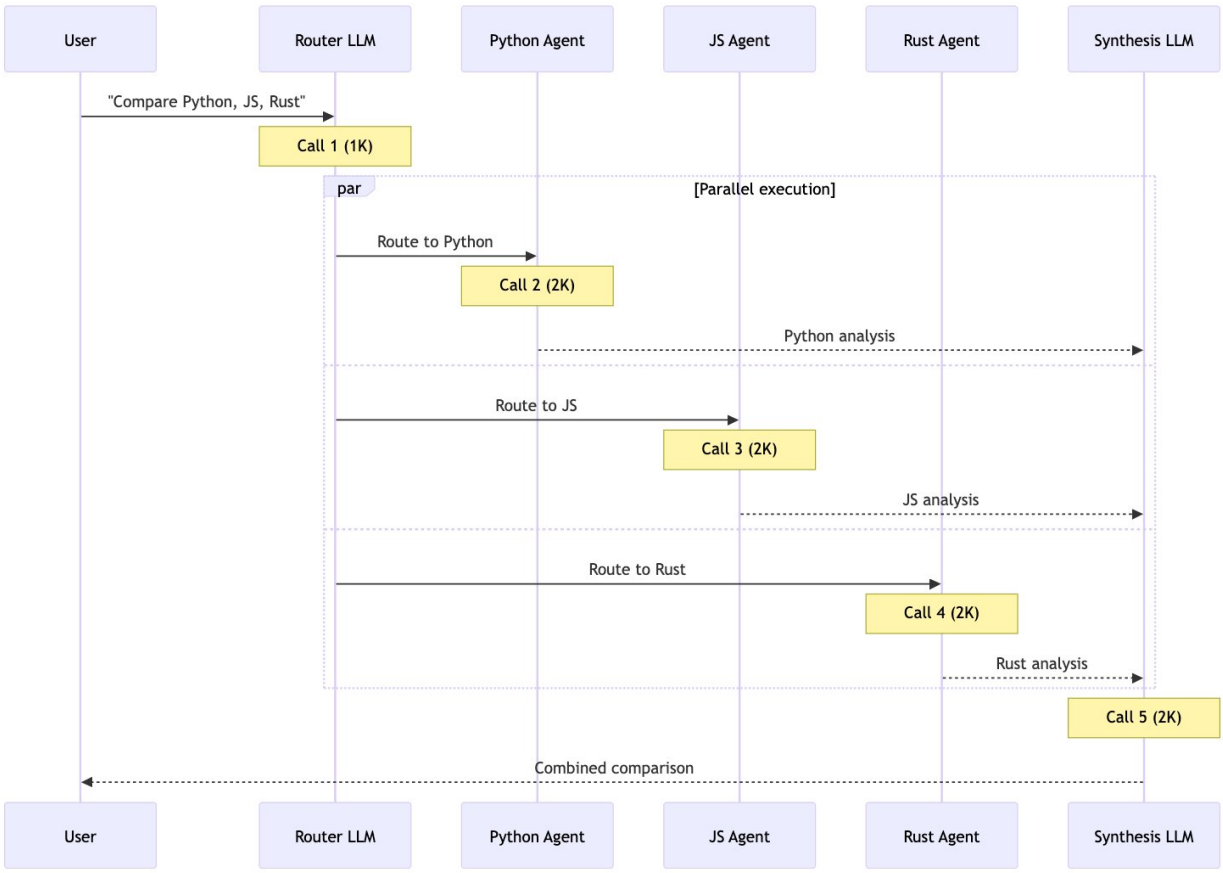


Figure 12: Router multi-domain: 5 calls with parallel execution